Project Report on

Hyperspectral Image Classification using Few Shot learning based on Prototypical Network

Submitted in partial fulfillment of the requirements of the degree of Bachelor in Engineering

By

Name of the student	Class	Roll No.
Adit Chheda	BE 9	15
Mayank Kantharia	BE 9	21
Aditya Shah	BE 9	38
Shivam Jadhav	BE 9	17

Under the guidance of

Dr. Manimala Mahato

DEPARTMENT OF COMPUTER ENGINEERING SHAH AND ANCHOR KUTCHHI ENGINEERING COLLEGE

CHEMBUR, MUMBAI – 400088.

2022 – 2023

www.shahandanchor.com Tel: 022 2558 0854

Mahavir Education Trust's

SHAH & ANCHOR KUTCHHI ENGINEERING COLLEGE

Mahavir Education Trust Chowk, W.T. Patil Marg, Chembur, Mumbai 400 088

Affiliated to University of Mumbai, Approved by D.T.E. & A.I.C.T.E.

UG Programs Computer Engineering & Information Technology accredited by NBA for 3 years w.e.f. 1st July 2019

ISO 9001:2008 Certified

Certificate

This is to certify that the report of the project entitled

Hyperspectral Image Classification using Few Shot learning based on Prototypical Network

is a bonafide work of

Sr No.	Name of the student	Class	Roll No.
1	Adit Chheda	BE 9	15
2	Mayank Kantharia	BE 9	21
3	Aditya Shah	BE 9	38
4	Shivam Jadhav	BE 9	17

submitted to the

UNIVERSITY OF MUMBAI

during semester VIII in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER ENGINEERING.

(Dr. Manimala Mahato)

Guide

(Prof. Uday Bhave)

Head of Department

(Dr. Bhavesh Patel)

Principal

Approval for Project Report for B. E. Semester VIII

This project report entitled **Hyperspectral Image Classification using Few Shot learning based on Prototypical Network** by **Adit Chheda, Mayank Kantharia, Aditya Shah, and Shivam Jadhav** is approved for semester VIII in partial fulfilment of the requirement for the award of the degree of Bachelor of Engineering.

Examiners

1._____

2._____

Guide

1. Dr. Manimala Mahato

Date: 28th April, 2023

Place: SAKEC, Mumbai

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Name of the student	**Class**	**Roll No.**	**Signature**
Adit Chheda	BE 9	15	
Mayank Kantharia	BE 9	21	
Aditya Shah	BE 9	38	
Shivam Jadhav	BE 9	17	

Date: 28th April, 2023

Place: SAKEC, Mumbai

Attendance Certificate

28th April, 2023

To,
The Principal
Shah and Anchor Kutchhi Engineering College,
Chembur, Mumbai-88

Subject: Confirmation of Attendance

Respected Sir,

This is to certify that Final year (BE) students **Adit Chheda, Mayank Kantharia, Aditya Shah, and Shivam Jadhav** have duly attended the sessions on the day allotted to them during the period from 20/01/2023 to 21/04/2023 for performing the Project titled **Hyperspectral Image Classification using Few Shot learning based on Prototypical Network.**

They were punctual and regular in their attendance. Following is the detailed record of the student's attendance.

Attendance Record:

Date	**Adit Chheda**	**Mayank Kantharia**	**Aditya Shah**	**Shivam Jadhav**
	Present/Absent	**Present/Absent**	**Present/Absent**	**Present/Absent**
20/01/2023	Present	Present	Present	Present
27/01/2023	Present	Present	Present	Present
03/02/2023	Present	Present	Present	Present
10/02/2023	Present	Present	Present	Present
17/02/2023	Present	Present	Present	Present
24/02/2023	Present	Present	Present	Present
03/03/2023	Present	Present	Present	Present
10/03/2023	Present	Present	Present	Present
17/03/2023	Present	Present	Present	Present
24/03/2023	Present	Present	Present	Present
31/03/2023	Present	Present	Present	Present
21/04/2023	Present	Present	Present	Present

Signature and Name of External Guide

Abstract

One of the challenges in machine learning is to deal with small datasets. When we have only a few examples to train our model, we may end up with an overfitted model that does not generalize well to new data. This is especially true for Hyperspectral Image Classification, where collecting Hyperspectral Images is difficult and costly.

To overcome this problem, we use Few-Shot Learning techniques that can classify new data with minimal training samples. In this project, we apply Stable Prototypical Network (SPN), a state-of-the-art Few-Shot Learning method, to Hyperspectral Image Classification. We also enhance the performance of SPN by adding Temperature Scaling and Attention Layer to the network. Our main goal is to improve the accuracy of the model on Hyperspectral Image Classification tasks.

The rest of this report is organized as follows. Chapter 1 briefly introduces our project. Chapter 2 reviews related work. Chapter 3 is the SRS documentation. Chapter 4 is about the timeline of our project development. Chapters 5 discuss details of the proposed framework. Chapter 6 gives details about implementation of this project. Chapter 7 presents the analysis of this project and Chapter 8 draws the conclusion and paves way to the future work.

Acknowledgement

We are thankful to our college Shah and Anchor Kutchhi Engineering College for considering our project and extending help at all stages needed during our work of collecting information regarding the project.

We are deeply indebted to our Principal, **Dr. Bhavesh Patel,** and Head of Computer Department, **Mr. Uday Bhave,** giving us this valuable opportunity to do this project. We express our hearty thanks to them for their assistance without which it would have been difficult in finishing this project synopsis and project review successfully.

It gives us immense pleasure to express our deep and sincere gratitude to our project guide, **Dr. Manimala Mahato** for her kind help and valuable advice during the development of project synopsis and for her guidance and suggestions.

We convey our deep sense of gratitude to all teaching and non-teaching staff for their constant encouragement, support, and selfless help throughout the project work. It is a great pleasure to acknowledge the help and suggestion, which we received from the Department of Computer Engineering.

We wish to express our profound thanks to all those who helped us in gathering information about the project. Our families too have provided moral support and encouragement several times.

Regards,
Adit Chheda
Mayank Kantharia
Aditya Shah
Shivam Jadhav

Table of Contents

Certificate

Project Report Approval

Declaration

Attendance Certificate

Abstract vi

Acknowledgement vii

Table of Contents viii

List Of Figures x

List Of Tables xi

Introduction 1

Literature Survey 2

2.1 Survey of Existing System 2

2.2 Limitations of Existing System 6

2.3 Problem Statement and Objective 7

2.4 Scope 7

Software Requirement Specification (SRS) 8

3.1 Introduction 8

3.1.1 Purpose 8

3.1.2 Intended Audience and Reading Suggestions 8

3.1.3 Scope 8

3.1.4 References 8

3.2 Overall Description 9

3.2.1 Perspective 9

3.2.2 Operating Environment 9

3.2.3 Design and Implementation Constraints 9

3.2.4 Dependencies 10

3.3 External Interface Requirements 10

3.3.1 Hardware Interfaces 10

3.3.2 Software Interfaces 10

3.4 Other Nonfunctional Requirements 10

3.4.1 Performance Requirements 10

3.4.2 Security Requirements 10

Project Scheduling and Planning 12

4.1 Project Scheduling 12

4.2 Gantt Chart 13

Proposed System 14

5.1 Framework / Algorithm 14

5.2 Details of Hardware & Software 15

5.2.1 Hardware Requirements 15

5.2.2 Software Requirements 15

5.3 Design Details 15

5.4 Methodology 16

5.4.1 Module 1: Data Loading and Pre-Processing 16

5.4.2 Module 2: Model Construction 16

5.4.3 Module 3: Training Phase 16

5.4.4 Module 4: Tunning Phase 16

5.4.5 Module 5: Testing Phase 16

Implementation Details 17

Result and Analysis 25

Conclusion and Future Scope 28

Bibliography 29

Appendix 30

Research Paper 31

List Of Figures

Figure 4.1: Gantt Chart - 1 ..... **13**
Figure 4.2: Gantt Chart - 2 ..... **13**
Figure 5.1: System Architecture **15**
Figure 6.1: RGB vs HSI images **18**
Figure 6.2: HSI Ground Truth and HSI Band Sample **19**
Figure 6.3: Few-Shot Image Classification with Meta Learning **20**
Figure 6.4: SPN Architecture **21**
Figure 6.5: Temperature Scaling using SoftMax Activation Function **23**
Figure 7.1: SPN Results **25**

List Of Tables

Table 4.1: Project Scheduling.....12
Table 7.1: Results of SPN when used with Temperature Scaling vs SPN 25
Table 7.2: Results of SPN when used with Temperature Scaling for n-times = 5 26
Table 7.3: Results of SPN when used with Temperature Scaling for n-times = 10 26
Table 7.4: Results of SPN when used with Temperature Scaling and Attention Mechanism 27

Chapter 1

Introduction

Hyperspectral images are images that capture a large number of narrow and contiguous spectral bands for each pixel in the scene. They provide rich information about the chemical composition and physical properties of the objects in the image. Hyperspectral images can be represented as a three-dimensional data cube, where the two spatial dimensions are x and y, and the spectral dimension is λ.

Depending on the application, different hyperspectral imaging systems can be used to capture images in different spectral ranges, such as ultraviolet (UV), visible (VIS), near-infrared (NIR), short-wave infrared (SWIR), mid-wave infrared (MWIR), or long-wave infrared (LWIR). Hyperspectral imaging has many benefits for people in various fields and domains. For example, hyperspectral imaging can be used for environmental monitoring, such as detecting vegetation stress, soil contamination, water quality, or mineral mapping. Hyperspectral imaging can also be used for medical imaging, such as diagnosing skin diseases, detecting cancer cells, or monitoring wound healing. Furthermore, hyperspectral imaging can be used for land use and land cover (LULC) classification, such as identifying crops, forests, urban areas, or roads. Hyperspectral imaging can also be used for other applications, such as food quality inspection, forensic analysis, cultural heritage preservation, or military surveillance.

Hyperspectral image (HSI) classification is a phenomenal mechanism to analyze diversified land cover in remotely sensed hyperspectral images. The main challenge of HSI classification is to deal with the high dimensionality, low sample size, and complex nonlinear structure of the data.

In this major project, we propose a method to enhance the performance of Stable Prototypical Network, a model that classifies Hyperspectral Images. Our method involves adding Temperature Scaling and Attention Layer to the network, which improves the accuracy of the classification.

Chapter 2

Literature Survey

2.1 Survey of Existing System

Table 2.1: Survey of Existing Work

Sr. No.	Author/Title	Approach & Remarks
1	D. Pal, V. Bundele, B. Banerjee and Y. Jeppu, "SPN: Stable Prototypical Network for Few-Shot Learning-Based Hyperspectral Image Classification."	The authors present a new way to improve model confidence for HSI classification with few samples. They use DropBlock in the encoder and reduce prediction variance by passing the samples multiple times through the encoder. This makes the model learn better representations for classification. They combine variance loss and CE loss to make SPN, which makes the model more confident in predicting classes with few spectral–spatial samples. SPN can be used with any FSL method to make it more stable. SPN uses the parameter space well to make stable class prototypes and beats the current best FSL methods for HSI classification.
2	A. Jha, S. Bose and B. Banerjee, "GAF-Net: Improving the Performance of Remote Sensing Image Fusion using Novel Global Self and Cross Attention Learning."	This paper introduces GAF-Net, a new multimodal fusion architecture for RS data. GAF-Net uses attention distillation to learn cross-attention between different modalities, and single-head spectral-spatial self-attention to learn features for each modality. The spectral attention captures both local and global channel contexts, which improves the channel features for RS data. GAF-Net outperforms the modality-specific classifiers by a large margin, even

		though it is shallower. The paper evaluates GAF-Net on five multimodal RS benchmark datasets with HSI, MSI, SAR, LiDAR, and audio data, and demonstrates its effectiveness.
3	Jake Snell, Kevin Swersky & Richard Zemel, "Prototypical Networks for Few-shot Learning."	Prototypical Networks is a simple and efficient method for few-shot learning that uses the mean of each class's examples in a learned representation space as a prototype. The network is trained with episodic training to perform well in the few-shot setting. The method outperforms recent meta-learning approaches and can be further improved by using Matching Networks extensions, choosing an appropriate distance metric, and modifying the episodic learning procedure. The method can also be generalized to zero-shot learning and achieves state-of-the-art results on the CUB-200 dataset.
4	Byung Soo Ko, H. -G. Kim, Kyo-Joong Oh and H. -J. Choi, "Controlled dropout: A different approach to using dropout on deep neural network."	Controlled dropout is a novel technique that selectively drops units to create a structured matrix. The authors compared it with traditional dropout and found that their accuracy difference diminishes with more weight updates. They also found that controlled dropout works best with a dropout rate between 0.4 and 0.8 and less than three hidden layers on MNIST dataset, similar to traditional dropout.
5	Y. Fang et al., “Dimensionality reduction of hyperspectral images based on robust spatial information using locally linear embedding,” IEEE Geosci. Remote Sens.	In this study, an enhanced locally linear embedding (LLE) approach based on reliable spatial information (designated RSLLE) is developed for reducing the dimensionality of hyperspectral data. It investigates and fully considers the intricacy of the spatial data for LLE. Instead of using the distance between two specific target pixels, RSLLE uses a

	Lett., vol. 11, no. 10, pp. 1712–1716, Oct. 2014.	form of spectral-spatial distance while looking for spectral neighbors. The robustness of the spectral-spatial distance is then illustrated using two additional processes, spatial neighbor sorting, and spatial neighbor filtering. According to two classification experimental findings, the suggested RSLLE approach greatly outperforms existing LLE methods in terms of performance, and its spectral-spatial classification accuracy is on par with other recent methods.
6	C. Rodarmel and J. Shan, "Principal component analysis for hyperspectral image classification," Surv. Land Inf. Syst., vol. 62, no. 2, pp. 115–122, Jun. 2002.	In this paper, the use of principal component analysis as a preprocessing method for classifying hyperspectral pictures is investigated. The study made use of the HYDICE and AVIRIS hyperspectral data sets. An investigation of the information contents of the principal component picture bands is conducted after a brief explanation of the principal component analysis technique. This study showed that only the first few bands contain important information. About 70% of classifications are correctly made when the first few primary component photos are used. This paper makes a case for the effectiveness and value of preprocessing hyperspectral image classification with principal component analysis.
7	B. Tu, C. Zhou, X. Liao, G. Zhang, and Y. Peng, "Spectral–spatial hyperspectral classification via structural-kernel	When utilizing contextual information in HSI, this letter offers a unique spatial-spectral classification approach for hyperspectral images (HSIs) based on a structural-kernel collaborative representation (SKCR), which considers one weak assumption of the spatial neighborhood that the pixels in a super

	collaborative representation," IEEE Geosci. Remote Sens. Lett., vol. 18, no. 5, pp. 861–865, May 2021.	pixel belong to the same class. The following steps make up the suggested technique. First, self-adaptive areas for the HSI are built using a super pixel segmentation approach. Following that, using the density peak and K nearest neighbors, the structural information contained within each super pixel block is retrieved. Next, the exploitation of the spatial and spectral information is done separately using dual kernels. Finally, a support-vector machine classifier is constructed using the combined dual kernels. Even with a limited number of training samples, the suggested method demonstrated good classification performance for two commonly used real hyperspectral datasets due to the collaborative representation's careful consideration of the weak assumption of spatial neighborhood.
8	A. Sellami, A. B. Abbes, V. Barra, and I. R. Farah, "Fused 3-D spectral–spatial deep neural networks and spectral clustering for hyperspectral image classification," Pattern Recognit. Lett., vol. 138, pp. 594–600, Oct. 2020.	This paper presents fused 3-D spectral-spatial deep neural networks for hyperspectral image classification, a novel method for HSI classification. It provides an unsupervised band selection approach. This method will automatically find a collection of groups Ck, each of which contains spectral bands that are similar to one another. In addition, the model extracts spectral-spatial information from the various groupings of chosen bands to increase classification accuracy. Each group is assigned a 3-D CNN model, which is combined to increase classification accuracy. The primary benefit of the suggested approach is the preservation of the initial spectral-spatial properties through the automated selection of pertinent spectral bands, which enhances the categorization of HSI with a minimal

		number of labeled samples. The efficiency of the suggested technique is tested using experiments on the Indian Pines and Salinas datasets, two genuine HSIs. Results indicate that, when compared to several state-of-the-art methodologies, the suggested method obtains superior classification rates and competitively good performances.
9	Y. Chen, Z. Lin, X. Zhao, G. Wang, and Y. Gu, "Deep learning-based classification of hyperspectral data," IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens., vol. 7, no. 6, pp. 2094–2107, Jun. 2014.	For the first time, the categorization of hyperspectral data employs the idea of deep learning. Principle component analysis (PCA), deep learning architecture, and logistic regression are all included in the system. Specifically, stacked autoencoders try to get relevant high-level features as a deep learning architecture. The results of experiments using frequently used hyperspectral data show that classifiers created in this deep learning-based system perform competitively. Additionally, the proposed hybrid spectral-spatial deep neural network provides a new avenue for future study and demonstrates the enormous potential of deep learning-based approaches for precise hyperspectral data categorization.

2.2 Limitations of Existing System

- To learn a relation module that computes relation scores between the support and query embeddings, relation networks for HSI classification are introduced. However, the adaption of FSL is highly difficult because of the high intraclass and low intraclass variance of remotely sensed data in the presence of surrounding pixel scattering.
- To make up for their vast parameter space, deep learning models are seen from a different angle as being data-hungry. But getting training samples for HSI is not simple.
- Alternative approaches to obtaining the tradeoff between the lack of labels and the

complexity of the deep learning models for HSI classification are required.

2.3 Problem Statement and Objective

Problem Statement

Hyperspectral Image Classification faces the challenge of small datasets, which can result in overfitting and poor generalization to new data. To address this issue, we aim to apply and enhance a state-of-the-art Few-Shot Learning method, Stable Prototypical Network (SPN), with Temperature Scaling and Attention Layer to improve the accuracy of Hyperspectral Image Classification with minimal training samples.

Objectives

The following are the objectives of the project:

- Improve SPN using Temperature Scaling and Attention layer.
- Reduce the overfitting problem of the Few-Shot Learning (FSL) systems.
- To extract useful information from few training samples.
- Reducing the effect of 'curse of dimensionality' from Hyperspectral Images.

2.4 Scope

- Research and implementation of Few-Shot Learning techniques, specifically the prototypical network approach, for Hyperspectral Image Classification.
- Collection and preparation of a small dataset of Hyperspectral Images for training and testing the model.
- Development and optimization of a prototypical network model for Few-Shot Learning in Hyperspectral Image Classification.
- Evaluation of the model’s performance and accuracy in classifying Hyperspectral Images with minimal training samples.

Chapter 3

Software Requirement Specification (SRS)

3.1 Introduction

3.1.1 Purpose

A solid FSL technique called prototypical networks (PNs), a computationally quick metric learning method, is suggested. This method simultaneously reduces the real class prediction variance and the cross-entropy (CE) loss for the query samples. As a result, the generalization confidence of PN is dramatically increased. Our approach thoroughly explores the parameter space and produces stable class prototypes for superior FSL by repeatedly decreasing the prediction variance in an episode.

3.1.2 Intended Audience and Reading Suggestions

The target audience for this publication includes researchers, students studying computer-related fields, especially the deep learning domain, and software engineers who want to explore new research.

3.1.3 Scope

The purpose of this method is to enhance the prototypical network by utilizing the softmax function of temperature scaling and attention layer mechanism to increase the fitting of models on small training samples. To improve training by using PCA and Monte Carlo Dropout to reduce the overfitting of the models. It will enhance the precision of analyzing hyperspectral images used in agriculture, water quality monitoring, coastal zone monitoring, and carbon quantity estimation.

3.1.4 References

- https://www.geeksforgeeks.org/how-to-write-a-good-srs-for-your-project/
- Marc Barrowclift, Kyle Musal, Travis Dutko, Aaron Chapin, Corey Everitt, Andrew Orner, Jiaking Jin, Matthew Tornetta, Eric Nordstrom, Ivan “Frankie” Orrego. Advised By: Rachel Greenstadt, Jeff Salvage- Software Requirements Specifications of Worden: Stylometry Tool

Prepared

- Joan Teamleader, Paul Adams, Bobbie Baker, Charles Charlie- Software Requirements Specifications of Web Publishing System

3.2 Overall Description

3.2.1 Perspective

This research is developed with the intention of improving the deep learning concept of few-shot learning. It takes the help of many concepts of deep learning such as Temperature Scaling, Monte Carlo Dropout, Attention Mechanism, and Principal Component Analysis.

3.2.2 Operating Environment

Python: Python is an easy-to-learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming.

Jupyter Notebook: It is an open-source web application that you can use to create and share documents that contain live code, equations, visualizations, and text. Jupyter Notebook is maintained by the people at Project Jupyter. The core supported programming languages that it supports: are Julia, Python, and R. Jupyter ships with the IPython kernel, which allows you to write your programs in Python.

Google Colab: Colab notebooks integrate executable code and rich text, as well as graphics, HTML, LaTeX, and other elements, in a single document. Colab notebooks are saved in your Google Drive account when you create them.

3.2.3 Design and Implementation Constraints

- Users may access from any computer that has internet browsing capabilities and an internet connection.
- The scope of this study is limited to using a dataset with limited HSI images which contain few images with dimensions.
- The design of the model consists of the Prototypical Network with Attention Mechanism architecture as the backbone model.
- In the Prototypical Network, the softmax function of temperature scaling and attention mechanism is used.

- Principal Component Analysis and Monte Carlo Dropout are used to handling the curse of dimensionality.

3.2.4 Dependencies

- Tensorflow
- Sklearn
- Statistics
- Scipy

3.3 External Interface Requirements

3.3.1 Hardware Interfaces

- OS: Windows 7 and above operating system
- RAM: 8 GB and above
- Storage Required: 10 GB
- Processor: Intel i5 or Ryzen 5 and higher
- GPU: Recommended
- Internet Connectivity

3.3.2 Software Interfaces

- Google Colab
- Python

3.4 Other Nonfunctional Requirements

3.4.1 Performance Requirements

- GPU should pe used to run the code for better performance of the code.

3.4.2 Security Requirements

Following are the security requirements for our project:

- Always use a strong password and avoid using the same password on other platforms.
- Two-factor authentication adds an extra layer of security to your account. Google Colab supports two-factor authentication through Google Authenticator.
- Only grant access to those who need it. Avoid sharing your Colab notebook publicly or

with people you don't know.

- Use encrypted data and always check the data encryption policy of any third-party data sources you use. Monitor your account activity regularly, and log out of Colab when you're not using it.
- Do not run code from untrusted sources or code that you have not reviewed. Keep your software and operating system up to date, as this can help protect against known security vulnerabilities.

By following these security requirements, we ensured that our work on Google Colab is secure and our data remains private.

Chapter 4

Project Scheduling and Planning

4.1 Project Scheduling

Table 4.1: Project Scheduling

Description	**Start Date**	**End Date**	**Duration (In Days)**
Define Research Question	18/07/2022	01/08/2022	14
Literature Review	01/08/2022	22/08/2022	21
Finalize Project Title	22/08/2022	29/08/2022	7
Understand Project Methodology	29/08/2022	19/09/2022	21
Collect and Analyze Data	19/09/2022	06/10/2022	17
Execute Existing Code and Interpret Results	06/10/2022	28/10/2022	22
Project Review	28/10/2022	28/10/2022	1
Implement Temperature Scaling in SPN Network	20/01/2023	03/02/2023	14
Perform Testing on code and store the Results	03/02/2023	17/02/2023	14
Implement Attention Layer in SPN Network	17/02/2023	03/03/2023	14
Perform Testing on code and store the Results	03/03/2023	17/03/2023	14
Result Analysis	17/03/2023	31/03/2023	14
Draw modified Model Architecture	31/03/2023	21/04/2023	21
Finalize Project Report, Logbook and PPT	21/04/2023	28/04/2023	7

4.2 Gantt Chart

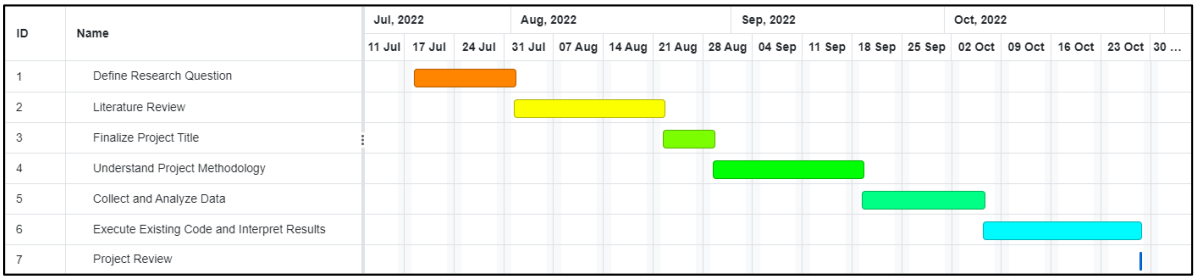

Figure 4.1: Gantt Chart - 1

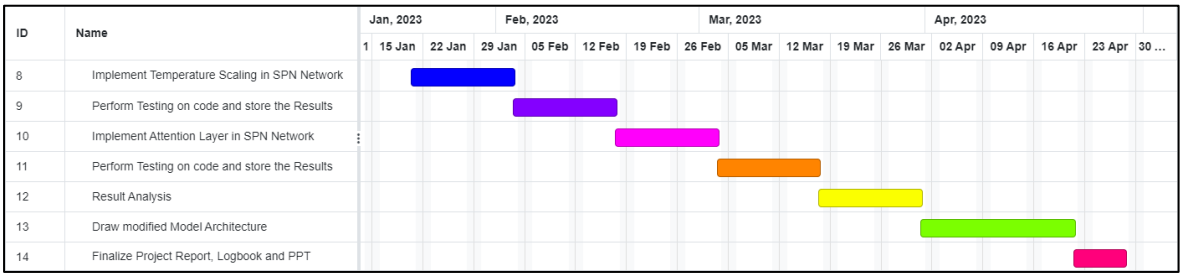

Figure 4.2: Gantt Chart - 2

Chapter 5

Proposed System

5.1 Framework / Algorithm

Algorithm for SPN Architecture with Attention Mechanism:

STEP 1: The model takes an input image with shape (im_height, im_width, im_depth, im_channel).

STEP 2: The input image is passed through a 3D convolutional layer with 8 filters and a kernel size of (3,3,7), followed by a DropBlock3D layer with a block size of 3 and a keep probability of 0.7.

STEP 3: The output from the previous step is passed through another 3D convolutional layer with 16 filters and a kernel size of (3,3,5), followed by another DropBlock3D layer with the same parameters as before.

STEP 4: The output from the previous step is passed through another 3D convolutional layer with 32 filters and a kernel size of (3,3,3).

STEP 5: The output from the previous step is reshaped and passed through a 2D convolutional layer with 64 filters and a kernel size of (3,3).

STEP 6: The output from the previous step is passed through two Attention layers: one with the default score mode and one with the score mode set to “concat”. Details of Hardware & Software

STEP 7: The output from step 4 is also passed through two Attention layers: one with the default score mode and one with the score mode set to “concat”. The output from this step is then reshaped and passed through another 2D convolutional layer with the same parameters as before.

STEP 8: The outputs from steps 6 and 7 are combined using another Attention layer and then added element-wise to the output from step 5.

STEP 9: The output from the previous step is normalized using a LayerNormalization layer.

STEP 10: The normalized output is flattened and passed through two Dense layers with 256 and 128 units respectively, separated by a Dropout layer with a rate of 0.4.

5.2 Details of Hardware & Software

5.2.1 Hardware Requirements:

- OS: Windows 7 and above operating system
- RAM: 8 GB and above
- Storage Required: 10 GB
- Processor: Intel i5 or Ryzen 5 and higher
- GPU: Recommended
- Internet Connectivity

5.2.2 Software Requirements:

- Google Colab
- Python

5.3 Design Details

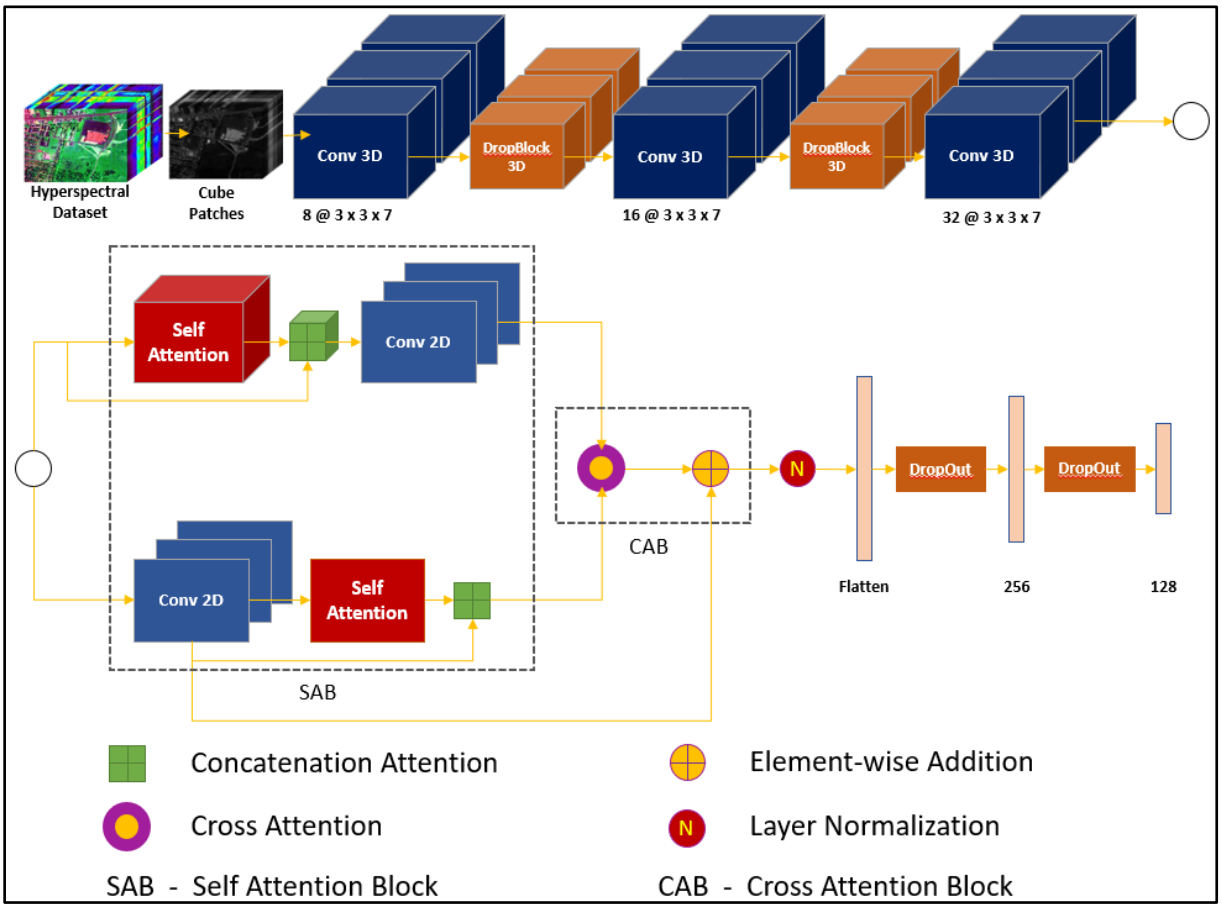

Figure 5.1 : System Architecture

5.4 Methodology

5.4.1 Module 1: Data Loading and Pre-Processing

- **Load Data:** Loading the HSI dataset, which includes data and labels.
- **Apply Principal Component Analysis (PCA) on HSI Data:** Perform PCA on a 3D numpy array by first reshaping it into a 2D array, performing PCA on the 2D array, and then reshaping the transformed data back into a 3D array. The PCA object is returned along with the transformed data.
- **Creating Image Cubes (Patches):** The output of PCA is taken as an input. Image cube is a sample of the dataset with a size of (window size * window size * No. of components of PCA). The sample is further used in the Patches Class.
- **Patches Class:** The image cubes are arranged according to the corresponding class of the dataset. It is further used to split the data into training data and testing data.
- **Data Loader:** Create a support set and query set using the output of Patches Class.

5.4.2 Module 2: Model Construction

The Patches are passed through various Neural Network Layers. These layers perform operations such as Convolution3D, DropBlock3D, Convolution2D, Attention, Normalization, Flatten, Dropout, and Dense. The output generated from the last layer is the Model.

5.4.3 Module 3: Training Phase

Prototypical Network uses the constructed Model, Adam's Optimizer, and Temperature Scaling using Softmax Activation Function to train the data.

5.4.4 Module 4: Tunning Phase

The number of Support Set and Query Set is reduced and passed into the Prototypical Network for Fine Tuning

5.4.5 Module 5: Testing Phase

Testing is performed on the test patches that were selected from Patches Class

Chapter 6

Implementation Details

Python Libraries Used:

- **Spectral**: A library for processing hyperspectral image data. It provides tools for reading, visualizing and analyzing such data.
- **Tensorflow**: An open-source library for machine learning and artificial intelligence. It provides tools for building and training neural networks and other machine learning models.
- **Random**: A built-in Python library for generating pseudo-random numbers. It provides functions for generating random numbers from various probability distributions.
- **Statistics**: A built-in Python library for calculating basic statistical measures such as mean, median, mode and standard deviation.
- **Sklearn**: A library for machine learning in Python. It provides tools for classification, regression, clustering and other common machine learning tasks.
- **Operator**: A built-in Python library that provides a set of functions corresponding to the intrinsic operators of Python. For example, operator.add(x, y) is equivalent to the expression x+y.
- **Plotly**: A library for creating interactive visualizations in Python. It supports a wide range of chart types and can be used in Jupyter notebooks or standalone web applications.
- **Numpy**: A library for numerical computing in Python. It provides a powerful N-dimensional array object and tools for working with arrays efficiently.
- **Matplotlib**: A library for creating static, animated and interactive visualizations in Python. It provides a wide range of plotting functions and customization options.
- **Scipy**: A library for scientific computing in Python. It provides tools for optimization, signal processing, linear algebra and other common scientific tasks.
- **Os**: A built-in Python library for interacting with the operating system. It provides functions for working with files and directories, launching other programs and more.

Hyperspectral Images:

As is common knowledge, the different hues of red, green, and blue are caused by the reflected light from objects falling within different wavelength ranges of the visible spectrum of electromagnetic radiation [long wavelengths, peaking near 564-580 nm (red), medium-wavelength, peaking near 534-545 nm (green), and short wavelength, peaking near 420-440 nm (blue)]. The visible spectrum only covers a small portion of the wavelengths that can be seen by human eyes, and many of these wavelengths are easily overlooked or invisible to them. As they say, "Color is in the eyes of the beholder," some colors or details that human eyes miss may be comparatively visible to the sight of other creatures.

Modern imaging technology has made it possible for us to collect and interpret data on wavelengths outside of the visible spectrum. Therefore, spectral imaging is broadly defined as the simultaneous capture of spatial and corresponding spectral information in image space, as well as their combination. It is somewhat comparable to an RGB color picture, but it contains many more channels and is thus difficult to perceive all at once.

Let's assume we have an n-band hyper-spectral image for comprehension reasons. This image has the dimensions n rows* m columns* n bands and is created by stacking n grey-scale photos (each band recording different light intensity data according to their wavelength values).

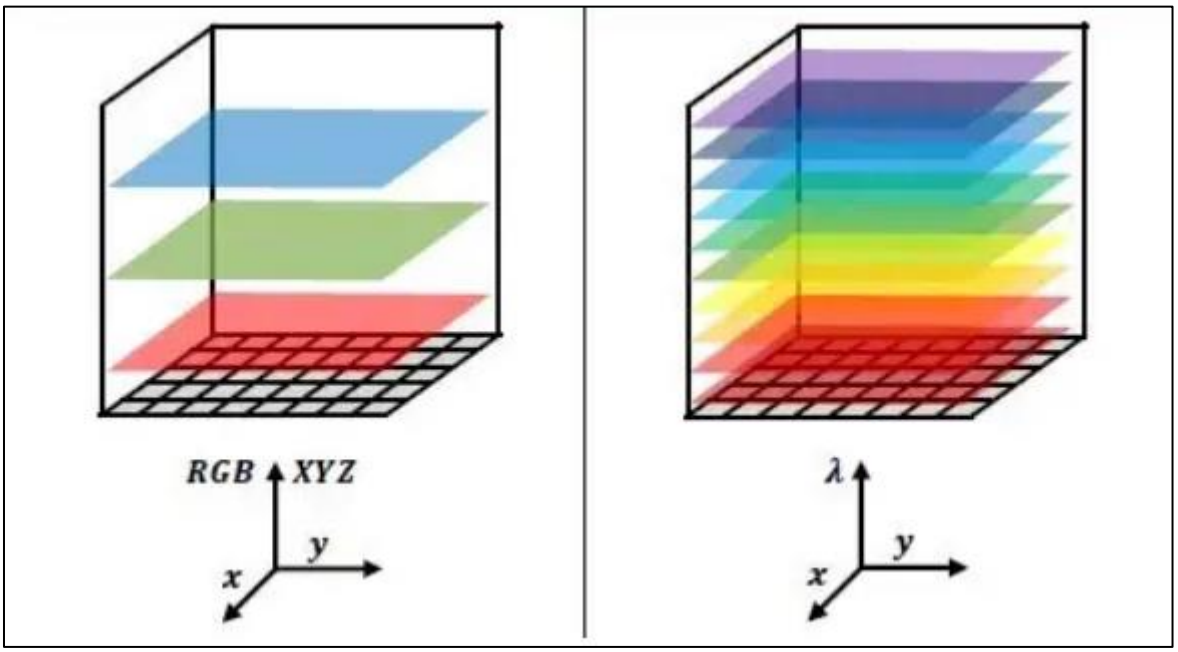

Figure 6.1: RGB vs HSI images

Multi-spectral photos, which are essentially n-band images but aren't always dispersed over a continuous wavelength range and contain fewer bands than the usual hyperspectral image, are another option before hyperspectral photographs. The data collected by satellites for remote sensing allows us to view actual multi-spectral photographs.

As a result, any HSI camera will produce an image that is in the shape of a hypercube and contains n-dimensional picture data, which can be difficult to handle at times. As a result, applying dimensional-reduction techniques to the same problem and having a variety of additional ways to treat the hypercube are prevalent. We could plot continuous spectrum data for this hyper-pixel if we tried to concentrate on just one of them.

It has uses in numerous industries, including the automotive industry, agriculture, textiles, and medicine. By visualizing data that is invisible to human eyes and figuring out how to make sense of it, HSI enables us to explore the unexplored.

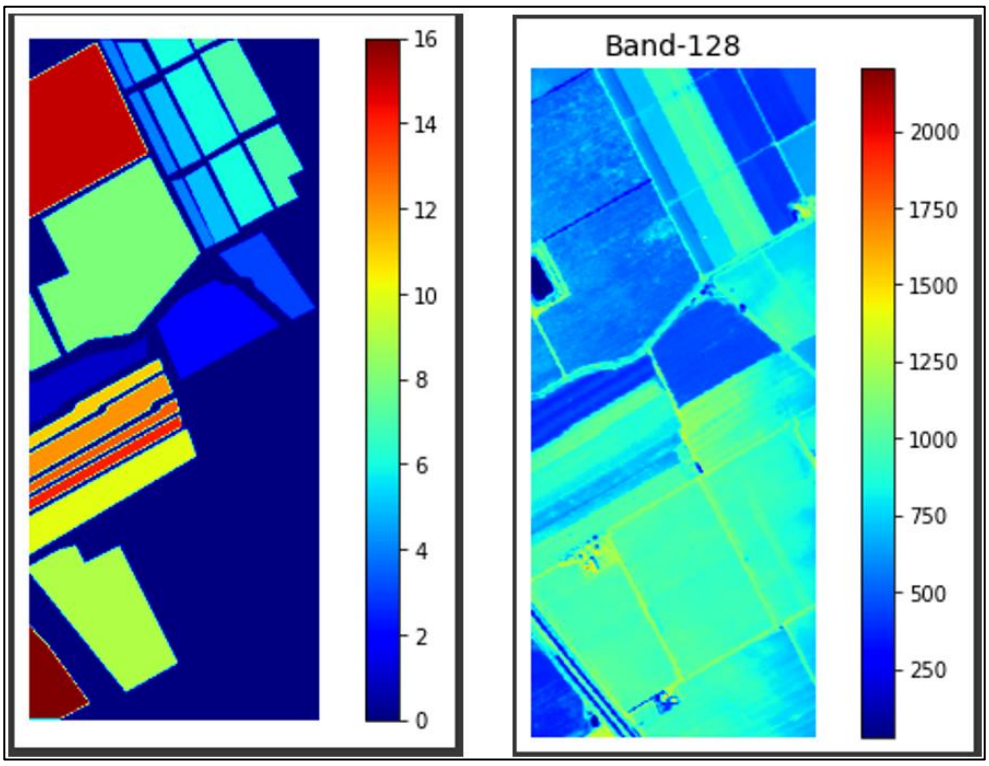

Figure 6.2: HSI Ground Truth and HSI Band Sample

High spatial resolution is a feature of the hyperspectral Salinas Scene dataset, which was acquired by the 224-band AVIRIS sensor over the Salinas Valley in California (3.7-meter pixels). The covered region consists of 217 samples along 512 lines. 20 water absorption bands, including [108-112], [154-167], and 224, were discarded. There was just at-sensor radiance data for this photograph. Vegetables, barren soils, and grape fields are all part of it. The ground truth

in Salinas has 16 classes.

Few-Shot Learning:

A branch of machine learning which involves categorizing new data when you only have a small number of training samples with supervised data is called few-shot learning. FSL is a very new field that needs more study and development. A computer vision model can perform well even with a small number of training examples. Consider the following scenario: We are in the healthcare industry and are having trouble classifying bone diseases from x-ray images. It's possible that the training set won't have enough photos for some uncommon disorders. This is precisely the kind of issue that an FSL classifier can address. An FSL task's first and most evident phase is to learn from previous, comparable issues. Hence, Few-Shot Learning is classified as a Meta-Learning issue.

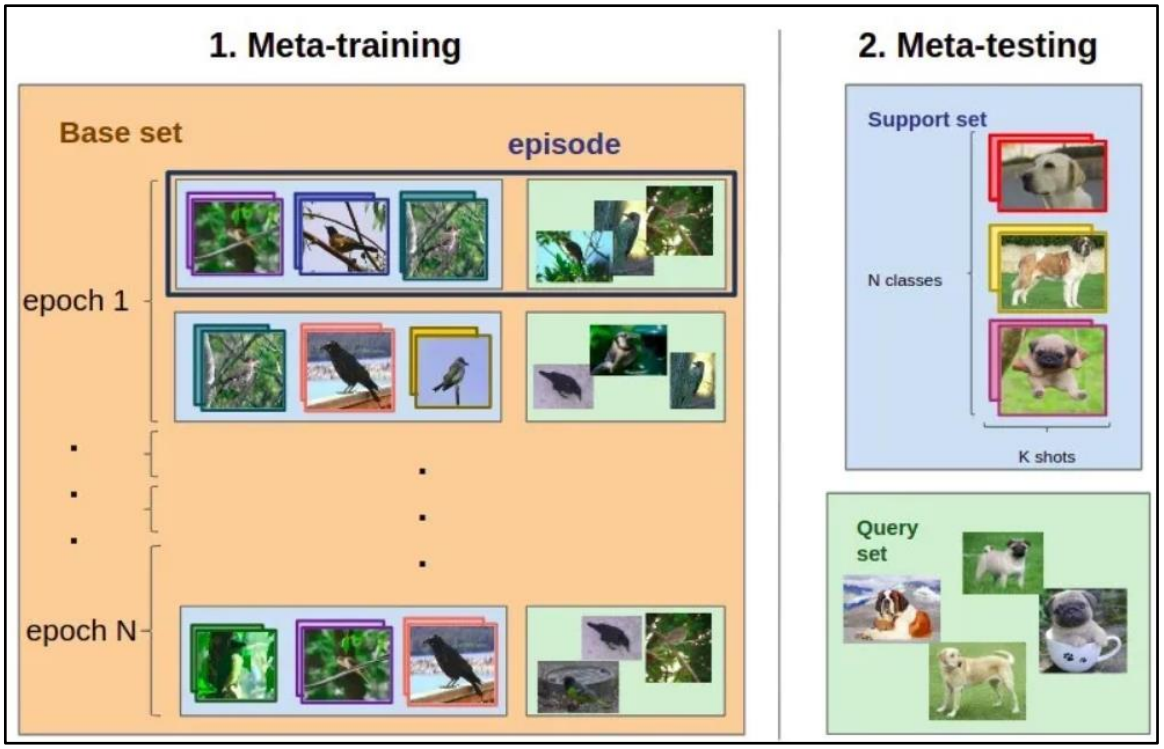

Figure 6.3: Few-Shot Image Classification with Meta Learning

An N-way K-Shot classification issue shall be defined. Consider the following:

1. A training (support) set that consists of:

a. **N** class labels

b. **K** labeled images for each class (a small amount, less than ten samples per class)

2. **Q** query images

We want to classify Q query images among the N classes. The N * K samples in the training set

are the only examples that we have. The main problem here is not enough training data.

SPN Architecture:

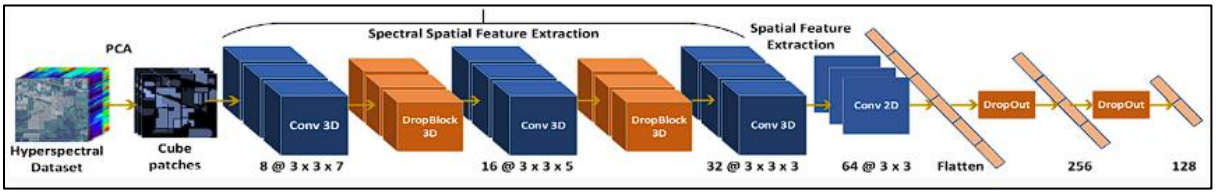

Figure 6.4: SPN Architecture

SPN describes a method for processing 3D hyperspectral image (HSI) data using a hybrid 3D and 2D convolutional neural network (CNN) architecture. The input data is first reduced in dimensionality using principal component analysis (PCA). The network then uses three successive 3D convolutional layers to learn spatial-spectral relationships, followed by a 2D convolutional layer to learn more diverse spatial features. The data is then flattened and passed through two fully connected layers. To prevent overfitting and learn structured spectral-spatial relationships, the model uses DropBlock between the Conv3D layers and Dropout between the FC layers. DropBlock applies a cubic mask of zeros to the dropout neuron positions, with parameters for block size and the number of features to be dropped.

Softmax Activation Function:

The softmax activation function is commonly used in the output layer of a neural network for multi-class classification problems. It takes a vector of real numbers as input and normalizes it into a probability distribution, where each element represents the probability of the corresponding class. The output of the softmax function is a vector of values between 0 and 1 that sum to 1. The softmax function is defined as follows: given an input vector x, the softmax function computes the exponential of each element e^{x_i} and normalizes it by dividing by the sum of all exponentials sum(e^{x}). The resulting vector y is given by

$$y_i = \frac{e^{x_i}}{\mathrm{sum}(e^{x})}$$

Where,

x = input vector

The softmax function is useful because it allows the model to output a probability distribution over the classes, which can be interpreted as the confidence of the model in its predictions. It also has desirable properties for training, such as being differentiable and having a well-defined gradient.

Softmax Activation with Temperature Scaling:

Temperature scaling is a technique used to calibrate the output probabilities of a neural network model. It involves dividing the logits (the inputs to the softmax function) by a temperature parameter T before applying the softmax function. Given an input vector x representing the logits, the temperature-scaled softmax function computes the exponential of each element $e^{\frac{x_i}{T}}$ and normalizes it by dividing by the sum of all exponentials sum ($e^{\frac{x}{T}}$). The resulting vector y is given by

$$y_i = \frac{e^{\frac{x_i}{T}}}{\mathrm{sum}(e^{\frac{x}{T}})}$$

Where,

x = Input Vector

T = Temperature Parameter

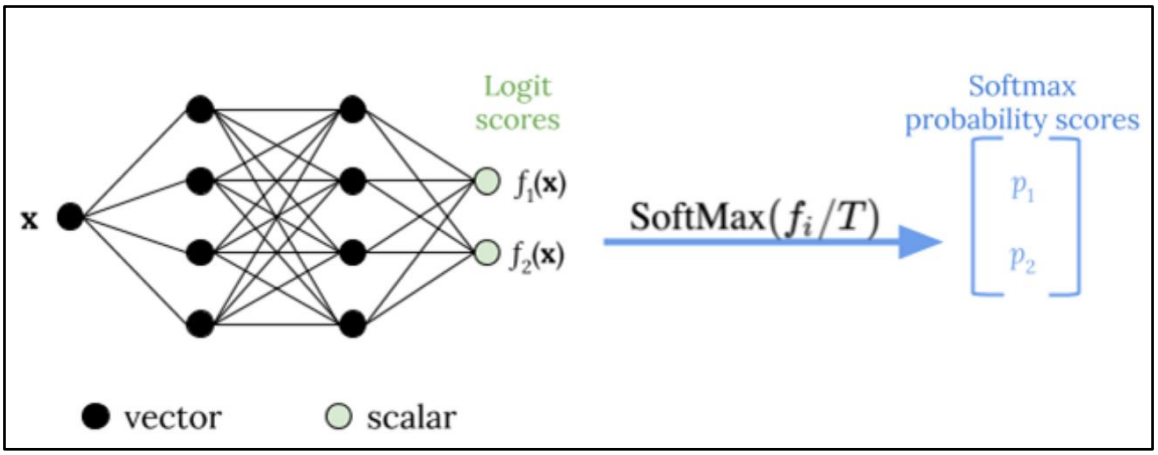

Figure 6.5: Temperature Scaling using SoftMax Activation Function

The temperature parameter T can be learned by minimizing the negative log-likelihood of the true labels on a validation set. This allows the model to output well-calibrated probabilities that reflect its uncertainty in its predictions.

Attention Mechanism:

Our implementation is of a 3D convolutional neural network (CNN) architecture (SPN) that uses attention mechanisms to improve feature extraction and classification performance on 3D volumetric data. The input to the model is a 5-dimensional tensor with dimensions corresponding to the height, width, depth, number of channels, and batch size of the input data.

The first layer in the model is a 3D convolutional layer that applies 8 filters with a kernel size of (3,3,7), ReLU activation, and some padding to the input tensor. This is followed by a DropBlock3D layer that randomly drops blocks of feature maps to reduce overfitting. The output of this layer is passed through another 3D convolutional layer with 16 filters and a kernel size of (3,3,5), followed by another DropBlock3D layer.

The next layer in the model is a 3D convolutional layer with 32 filters, a kernel size of (3,3,3), and ReLU activation. The output of this layer is reshaped into a 4D tensor and passed through a 2D convolutional layer with 64 filters and a kernel size of (3,3).

Two types of attention mechanisms are used in this model. The first attention mechanism is applied to the output of the 2D convolutional layer using the layers.Attention() function. This is followed by another attention mechanism with a concatenation operation applied to the outputs of the first attention layer and the 3D convolutional layer with 32 filters. The output of the second attention layer is reshaped into a 4D tensor and passed through another 2D convolutional layer

with 64 filters and a kernel size of (3,3). This whole mechanism is known as Self Attention Block.

The third and final attention mechanism is applied to the output of this layer and the output of the first attention mechanism using the layers.Attention() function with a sum operation. This mechanism is known as the Cross Attention Block.

Concatenation attention or additive attention:

$score(s_t, h_i) = v_a^T \tanh(W_a[s_t; h_i])$

Dot-product attention:

$score(s_t, h_i) = s_t^T h_i$

Where,

s_t = The query vector is typically derived from the current decoder state or output, and represents the context vector to which the input sequence elements will be compared

h_i = The key vector represents the encoded input sequence element at position 'i'.

s_t^T = The transpose of the query vector, which is necessary to ensure that the dot product is valid.

$s_t^T h_i$ = The dot product between the query vector and the key vector, which results in a scalar score that measures the similarity or relevance of the input sequence element to the current context.

v_a^T: The transpose of v_a, which has dimensions (1, hidden_size) and allows the dot product to be computed between the score and the context vectors.

The output of the final attention layer is passed through a layer normalization layer to normalize the output, flattened, and passed through two fully connected layers with 256 and 128 units, respectively. Finally, the output is passed through a dropout layer and a dense layer with a softmax activation function to obtain the class probabilities.

The model is trained using a categorical cross-entropy loss function and the Adam optimizer.

Chapter 7

Result and Analysis

All the results are based on the Indian Pines dataset. The results contain Overall Accuracy (OA), Kappa Accuracy (KA), and Average Accuracy (AA).

Result of Stable Prototypical Network (SPN):

n_{times}	Indian Pines		
	OA	KA	AA
1	95.33±0.23	93.55 ± 0.19	97.67 ± 0.29
5	98.16±0.11	97.43 ± 0.27	99.08 ± 0.14
10	98.33±0.17	97.66 ± 0.12	99.18 ± 0.12
15	98.49±0.13	97.16 ± 0.49	99.27 ± 0.12
20	99.16±0.09	98.82 ± 0.24	99.61 ± 0.05
25	**99.49±0.19**	**99.29 ± 0.12**	**99.75 ± 0.11**

Figure 7.1: SPN Results

In Figure 7.1, the original outputs for SPN architecture are displayed. It shows that n_{times} = 25 displays the most accurate results.

Results of SPN + Temperature Scaling:

Table 7.1: Results of SPN when used with Temperature Scaling vs SPN

Parameters		Temperature Scaling Values			Paper Values		
n_times	Tau Value	OA	AA	KA	OA	AA	KA
5	1.8	98.66	99	98.12	98.16±0.11	99.08±0.14	97.43±0.27
10	1.5	99.83	100	99.76	98.33±0.17	99.18±0.12	97.66±0.12

In Table 7.1, the results of SPN, when used with Temperature Scaling, are given. It shows the outputs for n_{times} = 5 and n_{times} = 10 for the original paper as well as SPN + Temperature Scaling. It also shows the T value for the results obtained.

Table 7.2: Results of SPN when used with Temperature Scaling for n-times = 5

T = 1.8						
Sr. No.	OA (in %)	Deviation	AA (in %)	Deviation	KA (in %)	Deviation
1	84.47	13.69	84	15.08	79.47	17.96
2	76.46	21.7	76	23.08	69.69	27.74
3	95.65	2.51	96	3.08	93.98	3.45
4	97.32	0.84	97	2.08	96.27	1.16
5	83.47	14.69	83	16.08	77.99	19.44
6	98.66	0.5	99	0.08	98.12	0.69
7	90.81	7.35	91	8.08	87.51	9.92
Average	89.54857143	8.754285714	89.42857143	9.651428571	86.14714286	11.48

In Table 7.2, outputs for n_{times} = 5 are shown. OA, KA, and AA are displayed. The value for T which gave the best result is 1.8.

Table 7.3: Results of SPN when used with Temperature Scaling for n-times = 10

T = 1.5						
Sr. No.	OA (in %)	Deviation	AA (in %)	Deviation	KA (in %)	Deviation
1	90.65	7.68	91	8.18	87.33	10.33
2	96.49	1.84	96	3.18	95.08	2.58
3	96.49	1.84	96	3.18	95.08	2.58
4	92.98	5.35	93	6.18	90.39	7.27
5	93.32	5.01	93	6.18	90.84	6.82
6	96.49	1.84	96	3.18	95.08	2.58
7	99.83	1.5	100	0.82	99.76	2.1
8	84.64	13.69	85	14.18	79.66	18
9	85.97	12.36	86	13.18	81.36	16.3
Average	92.98444444	5.678888889	92.88888889	6.473333333	90.50888889	7.617777778

In Table 7.3, outputs for n_{times} = 10 are shown. OA, KA, and AA are displayed. The value for T which gave the best result is 1.5.

Result of SPN + Temperature Scaling + Attention:

Table 7.4: Results of SPN when used with Temperature Scaling and Attention Mechanism

T = 1.8						
Sr. No.	OA (in %)	Deviation	AA (in %)	Deviation	KA (in %)	Deviation
1	96.66	1.5	97	2.08	95.36	2.07
2	99.33	1.17	99	0.08	99.06	1.63
3	96.82	1.34	97	2.08	95.59	1.84
4	97.66	0.5	98	1.08	96.74	0.69
5	99.49	1.33	99	0.08	99.29	1.86
6	99.83	1.67	100	0.92	99.76	2.33
7	100	1.84	100	0.92	100	2.57
8	95.49	2.67	95	4.08	93.77	3.66
Average	98.16	1.5025	98.125	1.415	97.44625	2.08125

Table 7.4, consists of outputs for n_{times} = 5 when SPN is used with Temperature Scaling and Attention Mechanism. The value of T is 1.8.

The above results show that the combination of SPN, Attention, and Temperature Scaling gives the best results and increases the performance by a significant amount.

Hence, it proves that our attention mechanism architecture using cross-attention and self-attention enhances the performance of the SPN architecture.

Chapter 8

Conclusion and Future Scope

The problem addressed in this work is the lack of reliable and efficient classification techniques for Few-Shot Learning (FSL) in Hyperspectral Imaging (HSI), especially for classes with limited spectral-spatial data.

In this work, we proposed a novel technique to enhance the reliability and confidence of Few-Shot Learning (FSL) models in Hyperspectral Image (HSI) classification. Our method involves incorporating Temperature Scaling into the encoder and reducing prediction variance by iteratively passing samples through the encoder. The architecture used as a backbone for Prototypical Network is developed according to Attention Mechanism that involves Cross Attention and Self Attention. By employing this Prototypical Network, we can effectively minimize variance and cross-entropy loss, thereby improving the model's accuracy in predicting classes with limited spectral-spatial data.

Our proposed strategy can be applied to any few-shot-based technique, resulting in more stable and robust models. Moreover, we have demonstrated that our approach outperforms the current state-of-the-art FSL techniques for HSI classification. The Prototypical Network not only optimizes the model's parameter space but also generates reliable class prototypes, thereby enhancing the overall performance of our model. Our findings have important implications for improving the accuracy and efficiency of FSL models in HSI classification, which could have significant practical applications in various domains such as remote sensing and medical imaging.

In conclusion, the proposed technique has the potential to revolutionize the field of FSL and improve the accuracy of classification tasks in various domains, including HSI. Future research can focus on exploring the effectiveness of this technique in other domains and extending it to other learning paradigms.

Chapter 9

Bibliography

[1] D. Pal, V. Bundele, B. Banerjee and Y. Jeppu, "SPN: Stable Prototypical Network for Few-Shot Learning-Based Hyperspectral Image Classification," *IEEE*, 2021 .

[2] S. B. a. B. B. A. Jha, "GAF-Net: Improving the Performance of Remote Sensing Image Fusion using Novel Global Self and Cross Attention Learning.".

[3] K. S. R. S. Z. Jake Snell, "Prototypical Networks for Few-shot Learning".

[4] B. Ko, H.-G. Kim, K.-J. Oh and H.-J. Choi, "Controlled dropout: A different approach to using dropout on deep neural network," *IEEE*, 2017.

[5] Y. Fang, H. Li, Y. Ma, K. Liang, Y. Hu, S. Zhang and H. Wang, "Dimensionality Reduction of Hyperspectral Images Based on Robust Spatial Information Using Locally Linear Embedding," *IEEE Geoscience and Remote Sensing Letters*, pp. 1712 - 1716, 2014.

[6] C. R. a. J. Shan, "Principal Component Analysis for Hyperspectral Image Classification," *Surveying and Land Information Systems*, pp. 115-000, 2002.

[7] C. Z. X. L. G. Z. a. Y. P. B. Tu, "Spectral–spatial hyperspectral classification via structural-kernel collaborative representation," *IEEE Geosci. Remote Sens*, p. 861–865, May 2021.

[8] A. B. A. V. B. a. I. R. F. A. Sellami, "Fused 3-D spectral– spatial deep neural networks and spectral clustering for hyperspectral image classification," *Pattern Recognit. Lett., vol. 138*, p. 594–600, Oct. 2020.

[9] Z. L. X. Z. G. W. a. Y. G. Y. Chen, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens., vol. 7, no. 6,*, p. 2094–2107, Jun. 2014.

Chapter 10

Appendix

Sr. No.	Event Type	Competition Details	Event Date	Venue
1.	Technical Paper	Paper Drafting	27/04/2023	Shah and Anchor Kutchhi Engineering College

Hyperspectral Image Classification using Few Shot learning based on Prototypical Network

Adit Chheda
Computer Engineering
Shah & Anchor Kutchhi Engineering College
Mumbai, India
aditbharatchheda@gmail.com

Mayank Kantharia
Computer Engineering
Shah & Anchor Kutchhi Engineering College
Mumbai, India
mayankkantharia01@gmail.com

Aditya Shah
Computer Engineering
Shah & Anchor Kutchhi Engineering College
Mumbai, India
adityashah539@gmail.com

Shivam Jadhav
Computer Engineering
Shah & Anchor Kutchhi Engineering College
Mumbai, India
shivam.jadhav_19@gmail.com

Dr. Manimala Mahato
Computer Engineering
Shah & Anchor Kutchhi Engineering College
Mumbai, India
manimala.mahato@sakec.ac.in

***Abstract*— One of the challenges in machine learning is to deal with small datasets. When we have only a few examples to train our model, we may end up with an overfitted model that does not generalize well to new data. This is especially true for Hyperspectral Image Classification, where collecting Hyperspectral Images is difficult and costly. To overcome this problem, we use Few-Shot Learning techniques that can classify new data with minimal training samples. In this project, we apply Stable Prototypical Network (SPN), a state-of-the-art Few-Shot Learning method, to Hyperspectral Image Classification. We also enhance the performance of SPN by adding Temperature Scaling and Attention Layer to the network. Our main goal is to improve the accuracy of the model on Hyperspectral Image Classification tasks.**

Keywords—image classification, hyperspectral images, stable prototypical network, few shot learning, temperature scaling, attention layer.

I. Introduction

Hyperspectral images are images that capture a large number of narrow and contiguous spectral bands for each pixel in the scene. They provide rich information about the chemical composition and physical properties of the objects in the image. Hyperspectral images can be represented as a three-dimensional data cube, where the two spatial dimensions are x and y, and the spectral dimension is λ.

Depending on the application, different hyperspectral imaging systems can be used to capture images in different spectral ranges, such as ultraviolet (UV), visible (VIS), near-infrared (NIR), short-wave infrared (SWIR), mid-wave infrared (MWIR), or long-wave infrared (LWIR). Hyperspectral imaging has many benefits for people in various fields and domains. For example, hyperspectral imaging can be used for environmental monitoring, such as detecting vegetation stress, soil contamination, water quality, or mineral mapping. Hyperspectral imaging can also be used for medical imaging, such as diagnosing skin diseases, detecting cancer cells, or monitoring wound healing. Furthermore, hyperspectral imaging can be used for land use and land cover (LULC) classification, such as identifying crops, forests, urban areas, or roads. Hyperspectral imaging can also be used for other applications, such as food quality inspection, forensic analysis, cultural heritage preservation, or military surveillance.

Hyperspectral image (HSI) classification is a phenomenal mechanism to analyze diversified land cover in remotely sensed hyperspectral images. The main challenge of HSI classification is to deal with the high dimensionality, low sample size, and complex nonlinear structure of the data.

In this major project, we propose a method to enhance the performance of Stable Prototypical Network, a model that classifies Hyperspectral Images. Our method involves adding Temperature Scaling and Attention Layer to the network, which improves the accuracy of the classification.

[1] The authors present a new way to improve model confidence for HSI classification with few samples. They use DropBlock in the encoder and reduce prediction variance by passing the samples multiple times through the encoder. This makes the model learn better representations for classification. This paper [2] introduces GAF-Net, a new multimodal fusion architecture for RS data. GAF-Net uses attention distillation to learn cross-attention between different modalities, and single-head spectral-spatial self-attention to learn features for each modality. [3] Prototypical Networks is a simple and efficient method for few-shot learning that uses the mean of each class's examples in a learned representation space as a prototype. The network is trained with episodic training to perform well in the few-shot setting. The method outperforms recent meta-learning approaches and can be further improved by using Matching Networks extensions, choosing an appropriate distance metric, and modifying the episodic learning procedure. [4] Controlled dropout is a novel technique that selectively drops units to create a structured matrix. The authors compared it with traditional dropout and found that their accuracy difference diminishes with more weight updates.

II. METHODOLOGY

A. *Algorithm for SPN Architecture with Attention Mechanism*

STEP 1: The model takes an input image with shape (im_height, im_width, im_depth, im_channel).

STEP 2: The input image is passed through a 3D convolutional layer with 8 filters and a kernel size of (3,3,7), followed by a DropBlock3D layer with a block size of 3 and a keep probability of 0.7.

STEP 3: The output from the previous step is passed through another 3D convolutional layer with 16 filters and a kernel size of (3,3,5), followed by another DropBlock3D layer with the same parameters as before.

STEP 4: The output from the previous step is passed through another 3D convolutional layer with 32 filters and a kernel size of (3,3,3).

STEP 5: The output from the previous step is reshaped and passed through a 2D convolutional layer with 64 filters and a kernel size of (3,3).

STEP 6: The output from the previous step is passed through two Attention layers: one with the default score mode and one with the score mode set to "concat". Details of Hardware & Software

STEP 7: The output from step 4 is also passed through two Attention layers: one with the default score mode and one with the score mode set to "concat". The output from this step is then reshaped and passed through another 2D convolutional layer with the same parameters as before.

STEP 8: The outputs from steps 6 and 7 are combined using another Attention layer and then added element-wise to the output from step 5.

STEP 9: The output from the previous step is normalized using a LayerNormalization layer.

STEP 10: The normalized output is flattened and passed through two Dense layers with 256 and 128 units respectively, separated by a Dropout layer with a rate of 0.4.

B. *Flow of Model Execution*

- Data Loading and Pre-Processing

1. Load Data: Loading the HSI dataset, which includes data and labels.
2. Apply Principal Component Analysis (PCA) on HSI Data: Perform PCA on a 3D numpy array by first reshaping it into a 2D array, performing PCA on the 2D array, and then reshaping the transformed data back into a 3D array. The PCA object is returned along with the transformed data.
3. Creating Image Cubes (Patches): The output of PCA is taken as an input. Image cube is a sample of the dataset with a size of (window size * window size * No. of components of PCA). The sample is further used in the Patches Class.
4. Patches Class: The image cubes are arranged according to the corresponding class of the dataset. It is further used to split the data into training data and testing data.
5. Data Loader: Create a support set and query set using the output of Patches Class.

- Model Construction

The Patches are passed through various Neural Network Layers. These layers perform operations such as Convolution3D, DropBlock3D, Convolution2D, Attention, Normalization, Flatten, Dropout, and Dense. The output generated from the last layer is the Model.

- Training Phase

Prototypical Network uses the constructed Model, Adam's Optimizer, and Temperature Scaling using Softmax Activation Function to train the data.

- Tunning Phase

The number of Support Set and Query Set is reduced and passed into the Prototypical Network for Fine Tuning.

- Testing Phase

Testing is performed on the test patches that were selected from Patches Class.

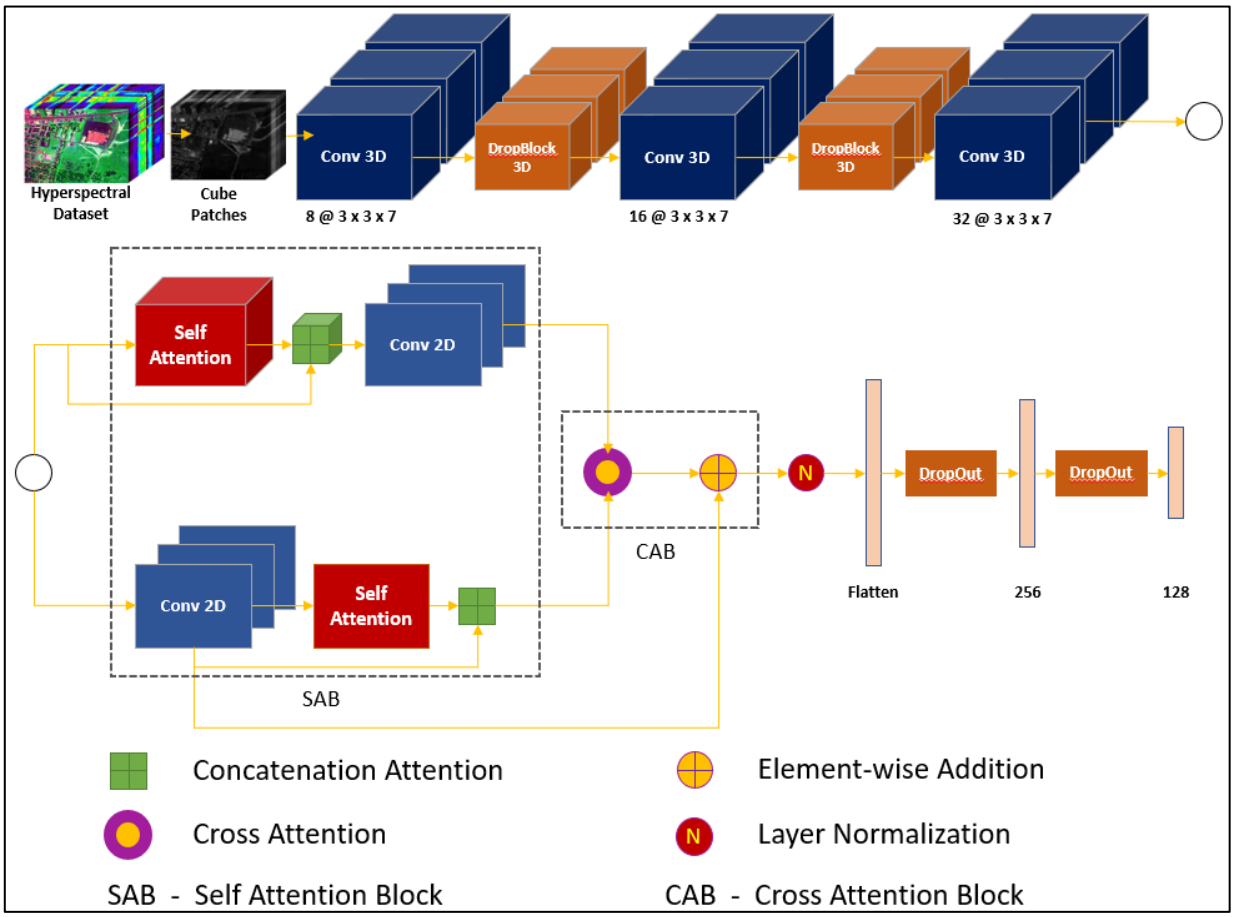

Fig.1 Attention Layer in SPN Architecture

C. *Working of SPN Architecture*

SPN describes a method for processing 3D hyperspectral image (HSI) data using a hybrid 3D and 2D convolutional neural network (CNN) architecture. The input data is first reduced in dimensionality using principal component analysis (PCA). The network then uses three successive 3D convolutional layers to learn spatial-spectral relationships, followed by a 2D convolutional layer to learn more diverse spatial features. The data is then flattened and passed through two fully connected layers. To prevent overfitting and learn structured spectral-spatial relationships, the model uses DropBlock between the Conv3D layers and Dropout between the FC layers. DropBlock applies a cubic mask of zeros to the dropout neuron positions, with parameters for block size and the number of features to be dropped.

The softmax activation function is commonly used in the output layer of a neural network for multi-class classification problems. It takes a vector of real numbers as input and normalizes it into a probability distribution, where each element represents the probability of the corresponding class. The output of the softmax function is a vector of values between 0 and 1 that sum to 1. The softmax function is defined as follows: given an input vector x, the softmax function computes the exponential of each element e^{x_i} and normalizes it by dividing by the sum of all exponentials sum(e^x). The resulting vector y is given by

$$y_i = \frac{e^{x_i}}{\mathrm{sum}(e^x)}$$

(1)

where,

x = input vector

The softmax function is useful because it allows the stable prototypical model to output a probability distribution over the classes, which can be interpreted as the confidence of the model in its predictions. It also has desirable properties for training, such as being differentiable and having a well-defined gradient.

D. *Implementing Temperature Scaling in SPN Architecture*

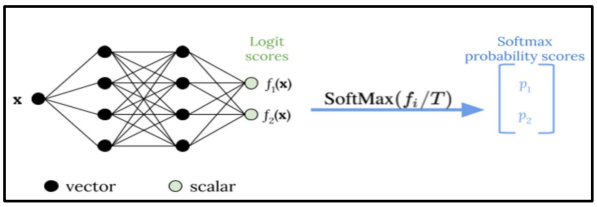

Fig.2 Temperature Scaling using SoftMax Activation Function

Temperature scaling is a technique used to calibrate the output probabilities of a neural network model. It involves dividing the logits (the inputs to the softmax function) by a temperature parameter T before applying the softmax function. Given an input vector x representing the logits, the temperature-scaled softmax function computes the exponential of each element $e^{\frac{x_i}{T}}$ and normalizes it by dividing by the sum of all exponentials sum $(e^{\frac{x}{T}})$. The resulting vector y is given by

$$y_i = \frac{e^{\frac{x_i}{T}}}{\text{sum}(e^{\frac{x}{T}})} \quad (2)$$

where,
x = Input Vector
T = Temperature Parameter

The temperature parameter T can be learned by minimizing the negative log-likelihood of the true labels on a validation set. This allows the stable prototypical model to output well-calibrated probabilities that reflect its uncertainty in its predictions.

E. Implementing Attention Mechanism in SPN Architecture

Our implementation is of a 3D convolutional neural network (CNN) architecture (SPN) that uses attention mechanisms to improve feature extraction and classification performance on 3D volumetric data. The input to the model is a 5-dimensional tensor with dimensions corresponding to the height, width, depth, number of channels, and batch size of the input data.

The first layer in the model is a 3D convolutional layer that applies 8 filters with a kernel size of (3,3,7), ReLU activation, and some padding to the input tensor. This is followed by a DropBlock3D layer that randomly drops blocks of feature maps to reduce overfitting. The output of this layer is passed through another 3D convolutional layer with 16 filters and a kernel size of (3,3,5), followed by another DropBlock3D layer.

The next layer in the model is a 3D convolutional layer with 32 filters, a kernel size of (3,3,3), and ReLU activation. The output of this layer is reshaped into a 4D tensor and passed through a 2D convolutional layer with 64 filters and a kernel size of (3,3).

Two types of attention mechanisms are used in this model. The first attention mechanism is applied to the output of the 2D convolutional layer using the layers.Attention() function. This is followed by another attention mechanism with a concatenation operation applied to the outputs of the first attention layer and the 3D convolutional layer with 32 filters. The output of the second attention layer is reshaped into a 4D tensor and passed through another 2D convolutional layer with 64 filters and a kernel size of (3,3). This whole mechanism is known as Self Attention Block.

The third and final attention mechanism is applied to the output of this layer and the output of the first attention mechanism using the layers.Attention() function with a sum operation. This mechanism is known as the Cross Attention Block.

Concatenation attention or additive attention:

$$score(s_t, h_i) = v_a^T \tanh(W_a[s_t; h_i]) \quad (3)$$

Dot-product attention:

$$score(s_t, h_i) = s_t^T h_i \quad (4)$$

where,
s_t = The query vector is typically derived from the current decoder state or output, and represents the context vector to which the input sequence elements will be compared.
h_i = The key vector represents the encoded input sequence element at position ‘i’.
s_t^T = The transpose of the query vector, which is necessary to ensure that the dot product is valid.
$s_t^T h_i$ = The dot product between the query vector and the key vector, which results in a scalar score that measures the similarity or relevance of the input sequence element to the current context.
v_a^T = The transpose of v_a, which has dimensions (1, hidden_size) and allows the dot product to be computed between the score and the context vectors.

The output of the final attention layer is passed through a layer normalization layer to normalize the output, flattened, and passed through two fully connected layers with 256 and 128 units, respectively. Finally, the output is passed through a dropout layer and a dense layer with a softmax activation function to obtain the class probabilities.

The model is trained using a categorical cross-entropy loss function and the Adam optimizer.

III. Result and Analysis

All the results are based on the Indian Pines dataset. The results contain Overall Accuracy (OA), Kappa Accuracy (KA), and Average Accuracy (AA).

A. Result of Stable Prototypical Network (SPN)

n_{times}	Indian Pines		
	OA	KA	AA
1	95.33±0.23	93.55 ± 0.19	97.67 ± 0.29
5	98.16±0.11	97.43 ± 0.27	99.08 ± 0.14
10	98.33±0.17	97.66 ± 0.12	99.18 ± 0.12
15	98.49±0.13	97.16 ± 0.49	99.27 ± 0.12
20	99.16±0.09	98.82 ± 0.24	99.61 ± 0.05
25	**99.49±0.19**	**99.29 ± 0.12**	**99.75 ± 0.11**

Fig.3 SPN Results

B. Results of SPN and Temperature Scaling

Parameters		Temperature Scaling Values			Paper Values		
n_times	Tau Value	OA	AA	KA	OA	AA	KA
5	1.8	98.66	99	98.12	98.16±0.11	99.08±0.14	97.43±0.27
10	1.5	99.83	100	99.76	98.33±0.17	99.18±0.12	97.66±0.12

Fig.4 Results of SPN when used with Temperature Scaling vs SPN

In Figure 4, the results of SPN, when used with Temperature Scaling, are given. It shows the outputs for n_{times} = 5 and n_{times} = 10 for the original paper as well as SPN + Temperature Scaling. It also shows the T value for the results obtained.

C. Results of SPN when used with Temperature Scaling for n_{times} = 5

T = 1.8						
Sr. No.	OA (in %)	Deviation	AA (in %)	Deviation	KA (in %)	Deviation
1	84.47	13.69	84	15.08	79.47	17.96
2	76.46	21.7	76	23.08	69.69	27.74
3	95.65	2.51	96	3.08	93.98	3.45
4	97.32	0.84	97	2.08	96.27	1.16
5	83.47	14.69	83	16.08	77.99	19.44
6	98.66	0.5	99	0.08	98.12	0.69
7	90.81	7.35	91	8.08	87.51	9.92
Average	89.54857143	8.754285714	89.42857143	9.651428571	86.14714286	11.48

Fig.5 Results of SPN when used with Temperature Scaling for n_{times} = 5

In Figure 5, outputs for n_{times} = 5 are shown. OA, KA, and AA are displayed. The value for T which gave the best result is 1.8.

D. Results of SPN when used with Temperature Scaling for n_{times} = 10

T = 1.5						
Sr. No.	OA (in %)	Deviation	AA (in %)	Deviation	KA (in %)	Deviation
1	90.65	7.68	91	8.18	87.33	10.33
2	96.49	1.84	96	3.18	95.08	2.58
3	96.49	1.84	96	3.18	95.08	2.58
4	92.98	5.35	93	6.18	90.39	7.27
5	93.32	5.01	93	6.18	90.84	6.82
6	96.49	1.84	96	3.18	95.08	2.58
7	99.83	1.5	100	0.82	99.76	2.1
8	84.64	13.69	85	14.18	79.66	18
9	85.97	12.36	86	13.18	81.36	16.3
Average	92.98444444	5.678888889	92.88888889	6.473333333	90.50888889	7.617777778

Fig.6 Results of SPN when used with Temperature Scaling for n_{times} = 10

In Figure 3, the original outputs for SPN architecture are displayed. It shows that n_{times} = 25 displays the most accurate results.

In Figure 6, outputs for n_{times} = 10 are shown. OA, KA, and AA are displayed. The value for T which gave the best result is 1.5.

E. Result of SPN + Temperature Scaling + Attention

T = 1.8						
Sr. No.	OA (in %)	Deviation	AA (in %)	Deviation	KA (in %)	Deviation
1	96.66	1.5	97	2.08	95.36	2.07
2	99.33	1.17	99	0.08	99.06	1.63
3	96.82	1.34	97	2.08	95.59	1.84
4	97.66	0.5	98	1.08	96.74	0.69
5	99.49	1.33	99	0.08	99.29	1.86
6	99.83	1.67	100	0.92	99.76	2.33
7	100	1.84	100	0.92	100	2.57
8	95.49	2.67	95	4.08	93.77	3.66
Average	98.16	1.5025	98.125	1.415	97.44625	2.08125

Fig.7 Results of SPN when used with Temperature Scaling and Attention Mechanism

Figure 7 consists of outputs for n_{times} = 5 when SPN is used with Temperature Scaling and Attention Mechanism. The value of T is 1.8.

The above results show that the combination of SPN, Attention, and Temperature Scaling gives the best results and increases the performance by a significant amount.

Hence, it proves that our attention mechanism architecture using cross-attention and self-attention enhances the performance of the SPN architecture.

IV. Conclusion

The problem addressed in this work is the lack of reliable and efficient classification techniques for Few-Shot Learning (FSL) in Hyperspectral Imaging (HSI), especially for classes with limited spectral-spatial data.

In this work, we proposed a novel technique to enhance the reliability and confidence of Few-Shot Learning (FSL) models in Hyperspectral Image (HSI) classification. Our method involves incorporating Temperature Scaling into the encoder and reducing prediction variance by iteratively passing samples through the encoder. The architecture used as a backbone for Prototypical Network is developed according to Attention Mechanism that involves Cross Attention and Self Attention. By employing this Prototypical Network, we can effectively minimize variance and cross-entropy loss, thereby improving the model's accuracy in predicting classes with limited spectral-spatial data.

Our proposed strategy can be applied to any few-shot-based technique, resulting in more stable and robust

models. Moreover, we have demonstrated that our approach outperforms the current state-of-the-art FSL techniques for HSI classification. The Prototypical Network not only optimizes the model's parameter space but also generates reliable class prototypes, thereby enhancing the overall performance of our model. Our findings have important implications for improving the accuracy and efficiency of FSL models in HSI classification, which could have significant practical applications in various domains such as remote sensing and medical imaging.

In conclusion, the proposed technique has the potential to revolutionize the field of FSL and improve the accuracy of classification tasks in various domains, including HSI. Future research can focus on exploring the effectiveness of this technique in other domains and extending it to other learning paradigms.

REFERENCES

[1] D. Pal, V. Bundele, B. Banerjee and Y. Jeppu, "SPN: Stable Prototypical Network for Few-Shot Learning-Based Hyperspectral Image Classification,"*IEEE,* 2021.

[2] S. B. a. B. B. A. Jha, "GAF-Net: Improving the Performance of Remote Sensing Image Fusion using Novel Global Self and Cross Attention Learning.".

[3] K. S. R. S. Z. Jake Snell, "Prototypical Networks for Few-shot Learning".

[4] B. Ko, H.-G. Kim, K.-J. Oh and H.-J. Choi, "Controlled dropout: A different approach to using dropout on deep neural network," *IEEE,* 2017.

[5] Y. Fang, H. Li, Y. Ma, K. Liang, Y. Hu, S. Zhang and H. Wang, "Dimensionality Reduction of Hyperspectral Images Based on Robust Spatial Information Using Locally Linear Embedding," *IEEE Geoscience and Remote Sensing Letters,* pp. 1712 - 1716, 2014.

[6] C. R. a. J. Shan, "Principal Component Analysis for Hyperspectral Image Classification," *Surveying and Land Information Systems,* pp. 115-000, 2002.

[7] C. Z. X. L. G. Z. a. Y. P. B. Tu, "Spectral–spatial hyperspectral classification via structural-kernel collaborative representation," *IEEE Geosci. Remote Sens,* p. 861–865, May 2021.

[8] A. B. A. V. B. a. I. R. F. A. Sellami, "Fused 3-D spectral– spatial deep neural networks and spectral clustering for hyperspectral image classification," *Pattern Recognit. Lett., vol. 138,* p. 594–600, Oct. 2020.

[9] Z. L. X. Z. G. W. a. Y. G. Y. Chen, "Deep learning-based classification of hyperspectral data," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens., vol. 7, no. 6,,* p. 2094–2107, Jun. 2014.